

Smart Irrigation System

Review: Final Demonstration

Team J : Byte Pioneers

Team Members

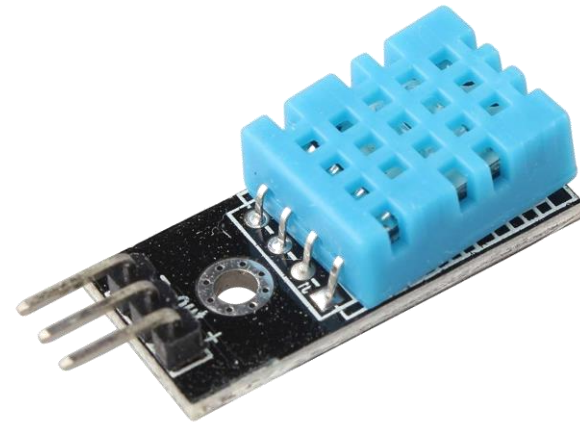
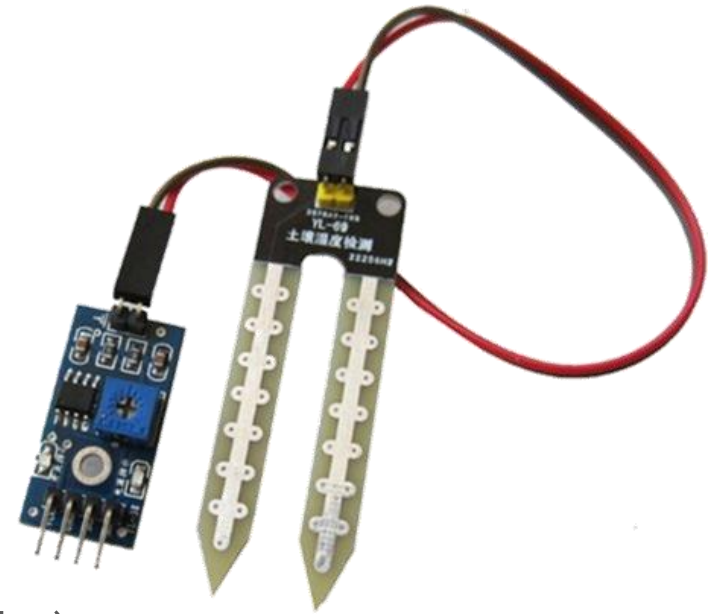
- SC21Bo89 – K Dhivakar
- SC21Bo93 – Rushikesh Gadekar
- SC21B115 – Sambit Saha
- SC21B116 – Samedh Kari
- SC21B117 – Shivam Kushwaha
- SC21B119 – Siddhant Meena
- SC21B126 – Utkarsh Utsav

Background

- Currently available sensors are based on ARM/Arduino, and quite costly.
- An application specific sensor for the same purpose.
- Affordable and power efficient.
- Moisture sensor in agricultural area with transmitter.
- Receiver along with temperature and humidity sensors and processor.
- Data received from different sensors get processed by the processor.

Sensors Used

- Moisture Sensor (SENo114)
- Temperature Humidity Sensor (DHT11)



Sensor Details

SEN0114

Specification	Value
Power Supply	3.3 V or 5 V
Output Voltage	0 – 4.2 V
Current	35 mA
Value Range	0 – 300: Dry Soil 300 – 700: Humid Soil 700 – 950: In Water

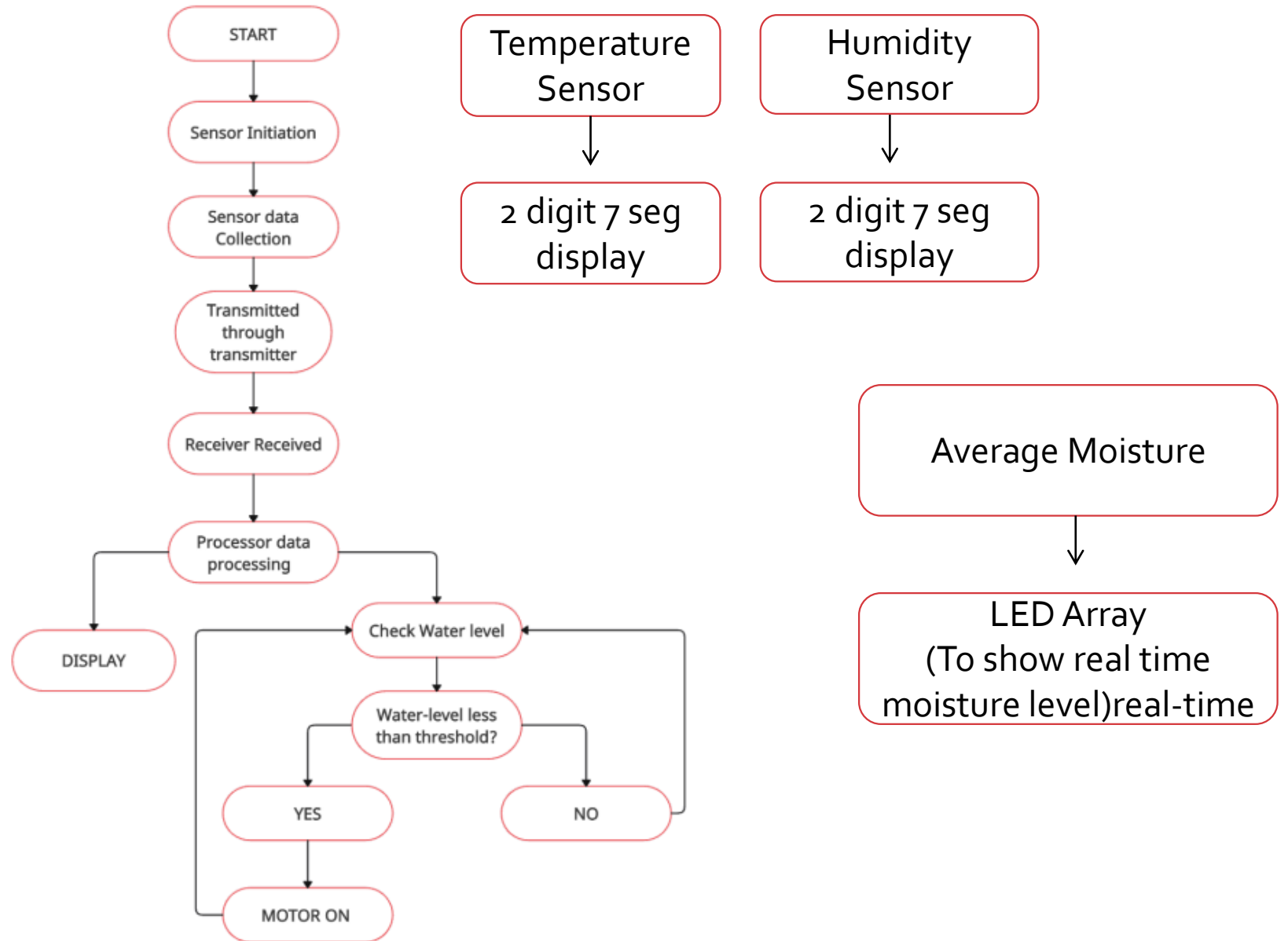
DHT11

Specification	Value
Voltage	5 V
Current (Avg)	0.2 – 1 mA
Accuracy	1 – 2 %
Resolution	8 bit

User Manual Format

[illegible]

Flowchart



Verilog Code

Memory Module

```
23 // Memory module
24 module MemoryModule(
25     input wire clk,
26     input wire [3:0] data_in,
27     input wire [4:0] address,
28     input wire write_enable,
29     output reg [3:0] data_out
30 );
31
32     reg [3:0] memory [31:0];
33     integer i=0;
34     always @(posedge clk) begin
35         if (write_enable) begin
36             for(i=0;i<31;i=i+1)
37                 begin
38                     memory[i+1]=memory[i];
39                 end
40             memory[0] <= data_in;
41         end
42         else
43             begin
44                 data_out <= memory[address];
45             end
46     end
47
48 endmodule
49
```

Main Module

```
51 // Main Special Purpose Processor module
52 module SpecialPurposeProcessor(
53     input wire clk,
54     input wire [3:0] soil_moisture,
55     input wire [3:0] temperature,
56     input wire [3:0] humidity,
57     input wire [3:0] threshold_knob,
58     output wire water_pump,
59     output wire [3:0] led_display,
60     output wire [7:0] seg_display1,
61     output wire [7:0] seg_display2
62 );
63
64 // Internal signals
65 reg [3:0] average_moisture;
66 reg [5:0] sample_count;
67 reg [5:0] hour_count;
68 reg [31:0] total_moisture;
69 reg x;
70
71 // State machine states
72 parameter IDLE = 2'b00;
73 parameter COLLECT_SAMPLES = 2'b01;
74 parameter COMPUTE_AVERAGE = 2'b10;
75
```

```
76 reg [1:0] state;
77 always @(posedge clk) begin
78     case (state)
79     IDLE:
80         if (sample_count == 64) begin
81             state <= COMPUTE_AVERAGE;
82         end else begin
83             state <= COLLECT_SAMPLES;
84         end
85     COLLECT_SAMPLES:
86         state <= IDLE;
87     COMPUTE_AVERAGE:
88         state <= IDLE;
89     endcase
90 end
91
92 // Memory module instantiation
93 MemoryModule memory_module(
94     .clk(clk),
95     .data_in(average_moisture),
96     .address(hour_count),
97     .write_enable(state == COMPUTE_AVERAGE),
98     .data_out()
99 );
100
```

Computing Average

```
100
101 // Logic to collect samples
102 always @(posedge clk) begin
103     if (state == COLLECT_SAMPLES) begin
104         total_moisture <= total_moisture + soil_moisture;
105         sample_count <= sample_count + 1;
106     end
107 end
108
109 // Logic to compute average and update memory
110 always @(posedge clk) begin
111     if (state == COMPUTE_AVERAGE) begin
112         average_moisture <= total_moisture >> 6; // Divide by 64
113         hour_count <= hour_count + 1;
114         sample_count <= 0;
115         total_moisture <= 0;
116     end
117 end
118
119 // Logic to predict water level and control pump
120 always @(posedge clk) begin
121     if (average_moisture < threshold_knob) begin
122         x <= 1;
123     end else begin
124         x <= 0;
125     end
126 end
127
128 assign water_pump = x;
129
130 // Display average moisture on LEDs
131 assign led_display = average_moisture;
132
```

Displaying the temperature on 7 seg display

```
133 //Display Temperature on seven segment
134
135 reg display_out1;
136 reg display_out2;
137
138 always @(*) begin
139     case (temperature)
140     4'b0000:
141     begin
142         display_out1 = 4'b0000;
143         display_out2 = 4'b0010;
144     end
145     4'b0001:
146     begin
147         display_out1 = 4'b0001;
148         display_out2 = 4'b0010;
149     end
150     4'b0010:
151     begin
152         display_out1 = 4'b0010;
153         display_out2 = 4'b0010;
154     end
155     4'b0011:
156     begin
157         display_out1 = 4'b0011;
158         display_out2 = 4'b0010;
159     end
160     4'b0100:
161     begin
162         display_out1 = 4'b0100;
163         display_out2 = 4'b0010;
164     end
165     4'b0101:
166     begin
167         display_out1 = 4'b0101;
168         display_out2 = 4'b0010;
169     end
```

Displaying the temperature on 7 seg display

```
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204

4'b0110:
begin
    display_out1 = 4'b0110;
    display_out2 = 4'b0010;
end
4'b0111:
begin
    display_out1 = 4'b0111;
    display_out2 = 4'b0010;
end
4'b1000:
begin
    display_out1 = 4'b1000;
    display_out2 = 4'b0010;
end
4'b1001:
begin
    display_out1 = 4'b1001;
    display_out2 = 4'b0010;
end
4'b1010:
begin
    display_out1 = 4'b0000;
    display_out2 = 4'b0011;
end
4'b1011:
begin
    display_out1 = 4'b0001;
    display_out2 = 4'b0011;
end
4'b1100:
begin
    display_out1 = 4'b0010;
    display_out2 = 4'b0011;
end
```

```
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232

4'b1101:
begin
    display_out1 = 4'b0011;
    display_out2 = 4'b0011;
end
4'b1110:
begin
    display_out1 = 4'b0100;
    display_out2 = 4'b0011;
end
4'b1111:begin
    display_out1 = 4'b0101;
    display_out2 = 4'b0011;
end
endcase
end

wire [7:0]r1;
wire [7:0]r2;
seg7 seg1(display_out1,r1);
seg7 seg2(display_out2,r2);

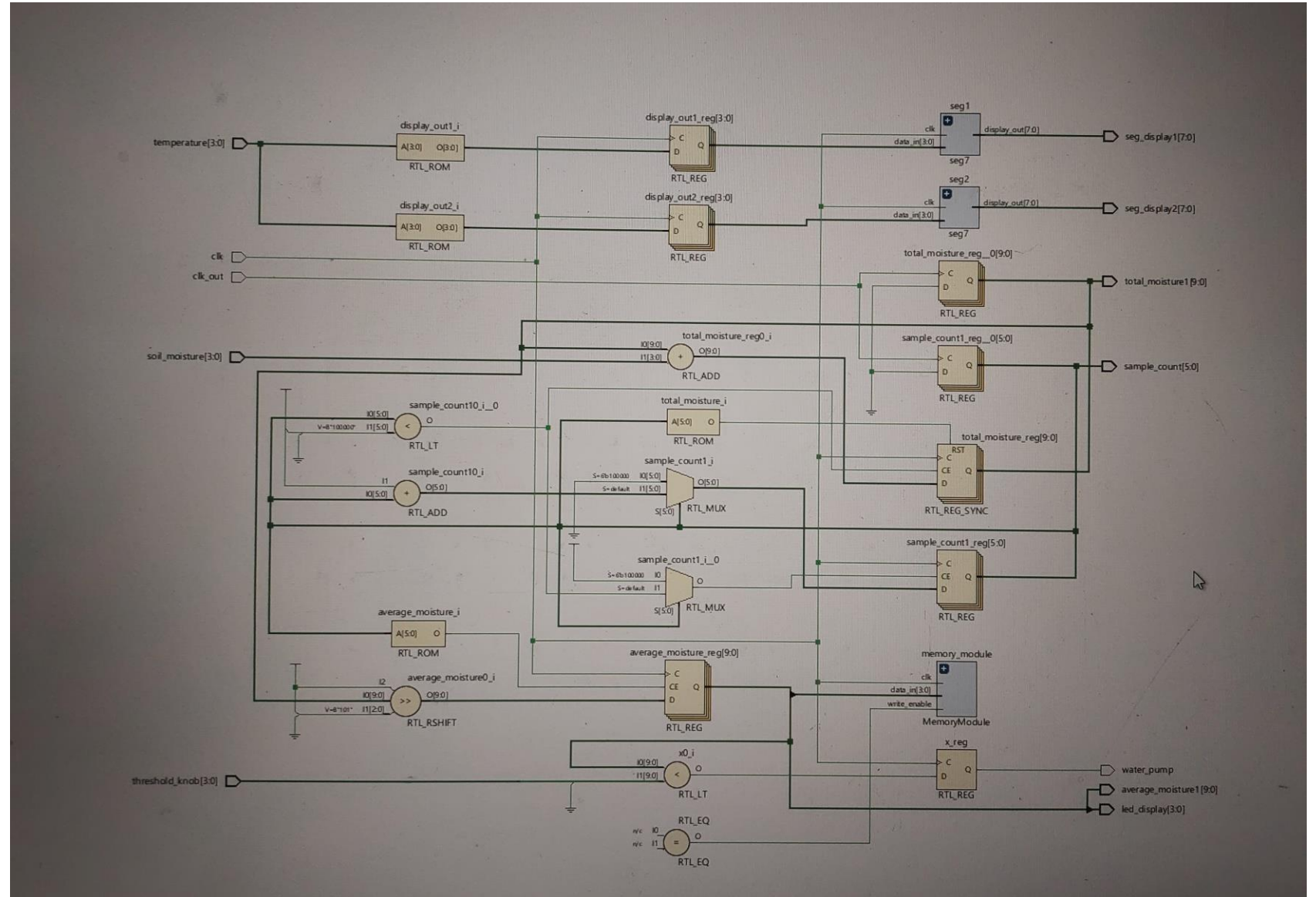
assign seg_display1 =r1;
assign seg_display2 =r2;

endmodule
```

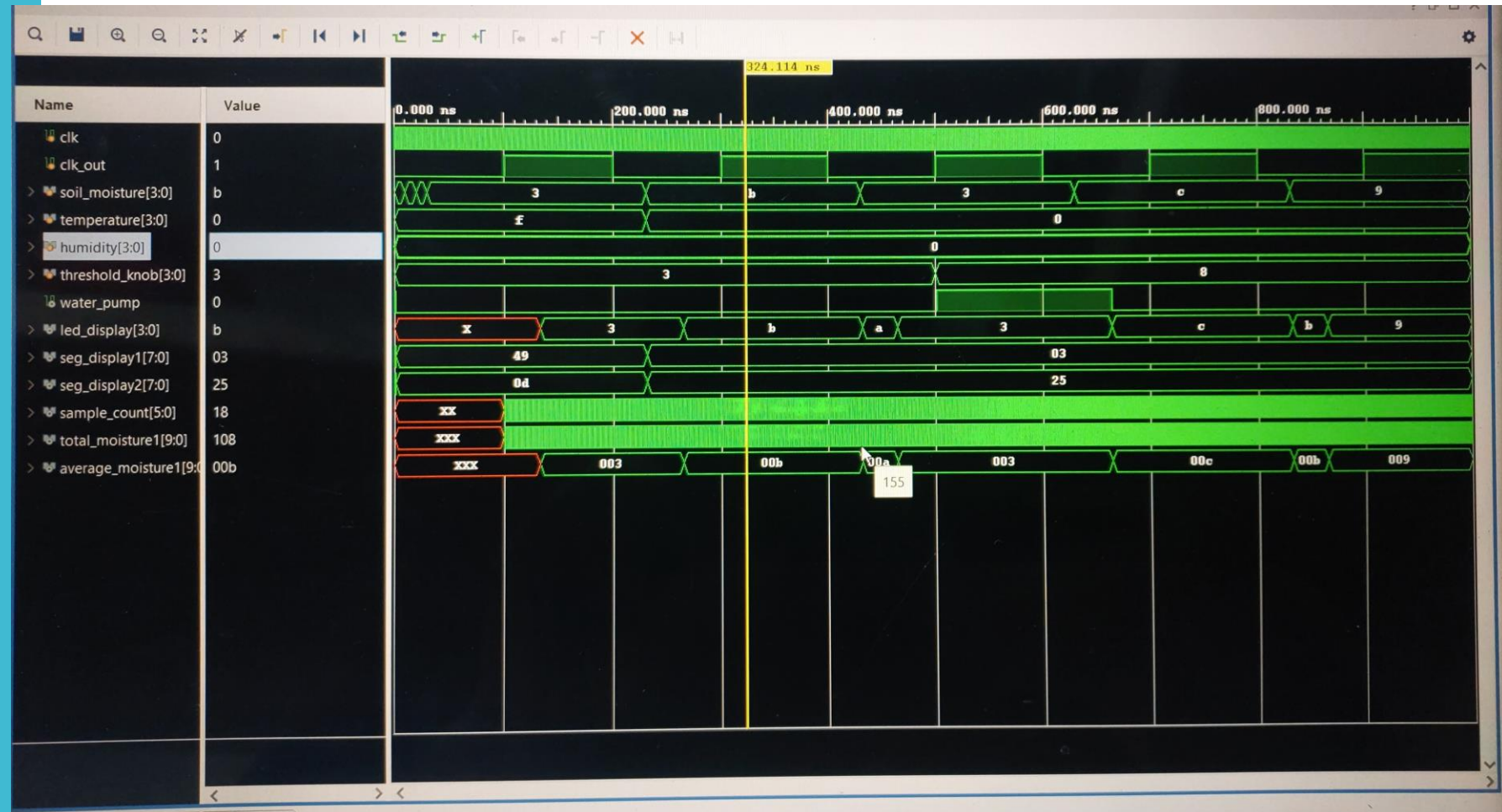
Module to display BCD data

```
236 module seg7(  
237     input [3:0]data_in,  
238     output reg [7:0]display_out  
239 );  
240  
241  
242 always @(*) begin  
243     case (data_in)  
244         4'b0000:  
245             display_out = 8'b00000011; //a,b,c,d,e,f,g,dot (zero)  
246         4'b0001:  
247             display_out = 8'b10011111; //one  
248         4'b0010:  
249             display_out = 8'b00100101; //two  
250         4'b0011:  
251             display_out = 8'b00001101; //three  
252         4'b0100:  
253             display_out = 8'b10011001; //four  
254         4'b0101:  
255             display_out = 8'b01001001; //five  
256         4'b0110:  
257             display_out = 8'b01000001; //six  
258         4'b0111:  
259             display_out = 8'b00011111; //seven  
260         4'b1000:  
261             display_out = 8'b00000001; //eight  
262         4'b1001:  
263             display_out = 8'b00001001; //nine  
264  
265     endcase  
266 end  
267 endmodule  
268
```

RTL Design



Simulation Results





Thank You